# LAPORAN TUGAS I

PENGEMBANGAN APLIKASI BERBASIS WEB (PABWE): PEMBANGUNAN APLIKASI WEB CATATAN BERBASIS BLOK MODULAR *SCRIPTORA* DENGAN MEMANFAATKAN *GENERATIVE ARTIFICIAL INTELLIGENCE* SEBAGAI ASISTEN PEMROGRAMAN.





| | | |
|---|---|---|
| Nama | : | Rafael Nobel Hutapea |
| NIM | : | 11S24040 |
| Mata Kuliah | : | Pemograman Berbasis Web (Pabwe) +P |
| Dosen Pengampu | : | ASD |

**PROGRAM STUDI INFORMATIKA**

**FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO**

**INSTITUT TEKNOLOGI DEL**


**TAHUN AJAR 2026/2027**

# BAB I PENDAHULUAN

## 1.1 Latar Belakang

Perkembangan *Generative Artificial Intelligence* (AI) dalam beberapa tahun terakhir telah membawa transformasi fundamental terhadap lanskap rekayasa perangkat lunak, khususnya dalam cara pengembang merancang arsitektur, menulis baris kode, melakukan pengujian (*testing*), serta memperbaiki kesalahan program (*debugging*). Kehadiran berbagai alat bantu berbasis AI generatif seperti Google Gemini, ChatGPT, Claude, dan GitHub Copilot memungkinkan pemrogram untuk memahami konsep pemrograman yang abstrak dengan lebih cepat, menghasilkan potongan kode (*boilerplate code*), mengidentifikasi letak anomali logika, menyusun struktur direktori aplikasi, serta memberikan rekomendasi optimasi kode secara instan.

Meskipun AI menawarkan efisiensi tinggi, ketergantungan penuh tanpa pemahaman dasar yang kuat dapat menimbulkan risiko teknis seperti kode yang tidak efisien, celah keamanan, atau logika program yang tidak sesuai dengan kebutuhan pengguna (*hallucination*). Oleh karena itu, dalam tugas mata kuliah Pengembangan Aplikasi Berbasis Web (PABWE) ini, dilakukan eksperimen pengembangan aplikasi web sederhana secara mandiri dengan menjadikan AI generatif sebagai asisten kolaboratif.

Proyek yang diangkat dalam proposal ini adalah **Scriptora**, sebuah aplikasi web pencatatan dan pengelolaan dokumen digital berbasis blok modular (*block-based note-taking app*) yang terinspirasi dari aplikasi produktivitas modern seperti Notion. Pembangunan *Scriptora* menuntut manipulasi *Document Object Model* (DOM) yang dinamis serta pengelolaan penyimpanan data di sisi klien (*client-side storage*). Melalui studi kasus pembangunan *Scriptora*, laporan ini mendokumentasikan secara menyeluruh bagaimana instruksi (*prompt*) diberikan kepada AI, bagaimana mengevaluasi setiap baris kode yang dihasilkan, mengidentifikasi *bug* yang muncul saat integrasi, serta menjelaskan keputusan teknis yang diambil sebagai bentuk tanggung jawab pengembang terhadap hasil akhir perangkat lunak.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang dibahas dalam proyek ini meliputi:

1. Bagaimana merancang antarmuka dan arsitektur aplikasi web pencatatan berbasis blok (*Scriptora*) yang responsif dan interaktif hanya dengan menggunakan teknologi web dasar (HTML5, CSS3, dan Vanilla JavaScript)?
2. Bagaimana menyusun struktur *prompt* yang efektif kepada *Generative AI* agar menghasilkan potongan kode yang bersih, terstruktur, dan mudah dipelihara?
3. Bagaimana melakukan verifikasi teknis, evaluasi kode, serta proses perbaikan (*debugging*) terhadap kesalahan logika yang dihasilkan oleh AI selama siklus pengembangan aplikasi?

## 1.3 Tujuan dan Manfaat Pengembangan

**Tujuan Pengembangan:**

1. Mengembangkan aplikasi web *Scriptora* yang berfungsi penuh untuk membuat, menyunting, mengelompokkan elemen blok teks, dan menghapus dokumen catatan secara lokal di peramban pengguna.
2. Mendemonstrasikan kemampuan berpikir kritis dan verifikasi teknis dalam memanfaatkan *Generative AI* sebagai asisten pemrograman selama waktu pengerjaan satu minggu.
3. Mengelola kode sumber proyek dan dokumen proposal ke dalam sistem kontrol versi (*version control*) menggunakan repositori GitHub secara terstruktur.

**Manfaat Pengembangan:**

1. Memberikan pengalaman praktis dalam mengintegrasikan logika manipulasi DOM tingkat lanjut dan pemanfaatan API localStorage pada pengembangan web.
2. Menciptakan alur kerja (*workflow*) kolaborasi manusia-AI yang sehat, di mana mahasiswa tetap memegang kendali penuh atas arsitektur dan validasi kualitas program.

# BAB II ANALISIS KEBUTUHAN DAN PERANCANGAN SISTEM

## 2.1 Deskripsi Umum Aplikasi *Scriptora*

*Scriptora* adalah aplikasi *Single Page Application* (SPA) sederhana berbasis web yang memungkinkan pengguna mengorganisasi ide, catatan kuliah, maupun daftar tugas harian dalam satu ruang kerja minimalis. Berbeda dengan aplikasi catatan konvensional yang hanya menyediakan satu kotak teks (*textarea*) statis, *Scriptora* mengadopsi pendekatan **modular berbasis blok**. Setiap paragraf teks, subjudul (*heading*), maupun daftar centang (*to-do list*) diperlakukan sebagai elemen blok independen yang dapat ditambah, diubah tipenya, atau dihapus secara dinamis.

## 2.2 Spesifikasi Kebutuhan Sistem

| Kode | Kategori | Nama Fitur | Deskripsi Fungsionalitas |
|---|---|---|---|
| **FR-01** | Manajemen Halaman | Buat & Pilih Dokumen | Pengguna dapat membuat halaman catatan baru melalui *sidebar* dan berpindah antar halaman secara instan tanpa memuat ulang (*reload*) peramban. |
| **FR-02** | Manajemen Halaman | Hapus Dokumen | Pengguna dapat menghapus halaman catatan yang sudah tidak diperlukan beserta seluruh blok di dalamnya. |
| **FR-03** | Editor Berbasis Blok | Tambah Blok Konten | Pengguna dapat menambahkan blok konten baru ke dalam halaman aktif dengan pilihan tipe: *Heading 1*, *Paragraf*, atau *To-Do Checklist*. |
| **FR-04** | Editor Berbasis Blok | Sunting Konten *Real-time* | Setiap perubahan teks di dalam blok langsung diperbarui pada *state* aplikasi menggunakan atribut contenteditable atau elemen input dinamis. |
| **FR-05** | Penyimpanan Data | *Auto-Save LocalStorage* | Seluruh perubahan struktur halaman dan isi blok disimpan secara otomatis ke dalam window.localStorage peramban agar data bersifat persisten. |
| **FR-06** | Antarmuka (UI) | *Dark / Light Mode* | Pengguna dapat mengubah tema warna antarmuka aplikasi antara mode terang dan mode gelap untuk kenyamanan membaca. |

## 2.3 Arsitektur Teknis dan Struktur Data

Aplikasi ini dibangun tanpa menggunakan *framework* eksternal (*pure Vanilla JS*) untuk memastikan pemahaman mendalam terhadap konsep dasar manipulasi web. Struktur

penyimpanan data di dalam localStorage dirancang menggunakan format JSON (*Array of Objects*) dengan skema sebagai berikut:

```
JSON
[
 {
  "id": "doc_1729150001",
  "title": "Catatan Kuliah PABWE Minggu 1",
  "updatedAt": "2026-10-17T14:30:00Z",
  "blocks": [
   {
    "blockId": "blk_101",
    "type": "h1",
    "content": "Pengenalan Generative AI dalam Web Dev",
    "checked": false
   },
   {
    "blockId": "blk_102",
    "type": "todo",
    "content": "Membuat repositori GitHub dan memberi akses ke dosen",
    "checked": true
   }
  ]
 }
]
```

### 2.4 Struktur Direktori Repositori GitHub

Untuk memenuhi aspek penilaian kerapian repositori (30%), struktur berkas proyek diatur sebagai berikut:

```
scriptora-pabwe/
 ├──── index.html        # Kerangka utama antarmuka aplikasi (Sidebar & Editor Area)
 ├──── css/
 │     └──── style.css      # Styling layout Flexbox, tipografi, dan variabel tema (Dark/Light)
 ├──── js/
 │     ├──── storage.js     # Modul khusus untuk membaca dan menyimpan data ke localStorage
 │     └──── app.js         # Logika utama manipulasi DOM, manajemen state, dan event listener
 ├──── docs/
 │     └──── Proposal_Tugas1_PABWE_Rafael.pdf  # Dokumen proposal laporan tugas
 └──── README.md          # Dokumentasi repositori, cara menjalankan aplikasi, dan penjelasan
proyek
```

# BAB III IMPLEMENTASI DAN PROSES KOLABORASI DENGAN AI

Bab ini mendokumentasikan secara rinci proses interaksi dengan *Generative AI* mulai dari pemberian instruksi (*prompting*), hasil kode yang diberikan oleh AI, analisis kesalahan (*error identification*), hingga perbaikan yang dilakukan oleh pengembang.

## 3.1 Tahap 1: Perancangan Layout Antarmuka (HTML & CSS)

### A. Instruksi (*Prompt*) kepada AI

Pada tahap awal, AI diminta untuk membangun kerangka dasar halaman web dua kolom yang menyerupai ruang kerja Notion.

```
Saya sedang mengerjakan proyek aplikasi web bernama Scriptora (aplikasi catatan berbasis blok ala mini Notion) menggunakan HTML5, CSS3, dan Vanilla JavaScript murni tanpa framework apa pun.

Untuk tahap pertama, tolong buatkan file `index.html` dan `css/style.css` dengan ketentuan yang sangat ketat berikut:

### 1. Struktur Layout
- Gunakan layout Flexbox dengan tinggi 100vh.
- Body utama tidak boleh memiliki scroll vertikal. Hanya area konten editor yang boleh di-scroll.
- Sidebar kiri dengan lebar tetap 260px.
- Area editor di sebelah kanan yang mengisi sisa ruang (flex: 1).

### 2. Sidebar (Kiri)
Isi sidebar harus terdiri dari:
- Nama aplikasi "Scriptora" di bagian paling atas (bold, ukuran besar).
- Tombol "+ Halaman Baru".
- Tombol toggle Dark/Light Mode (dengan ikon atau teks yang jelas).
- Daftar halaman catatan (sementara boleh diisi dengan 2–3 item dummy sebagai contoh).

### 3. Area Editor (Kanan)
- Input judul halaman di bagian atas (ukuran besar, font tebal, contenteditable atau input text yang terlihat seperti judul Notion).
- Kontainer utama untuk blok-blok catatan yang dapat di-scroll secara vertikal (overflow-y: auto).
- Floating action bar di bagian bawah area editor yang berisi 3 tombol:
  - "+ Heading 1"
  - "+ Paragraf"
  - "+ To-Do"

### 4. Desain & Color Palette (WAJIB DIGUNAKAN)
Gunakan CSS Variables agar mendukung Dark Mode dan Light Mode dengan estetika minimalis mirip Notion.

Palette warna yang harus digunakan:

**Light Mode (default):**
- Background utama: #FFF4EB (Seashell)
- Background sidebar: #F6E0B6 (Wheat)
- Background kartu/blok: #FFFFFF atau sedikit lebih terang dari Seashell
```

- Teks utama: #3D1534 (Midnight Violet)
- Aksen/primer: #3E4B8E (French Blue)
- Secondary: #A6BCC9 (Powder Blue)
- Border/hover ringan: #A6BCC9

**Dark Mode:**
- Background utama: #3D1534 (Midnight Violet) atau sedikit lebih gelap
- Background sidebar: warna lebih gelap dari Midnight Violet
- Teks utama: #FFF4EB (Seashell)
- Aksen: #A6BCC9 (Powder Blue) dan #F6E0B6 (Wheat)
- Tombol aktif/hover: #3E4B8E (French Blue)

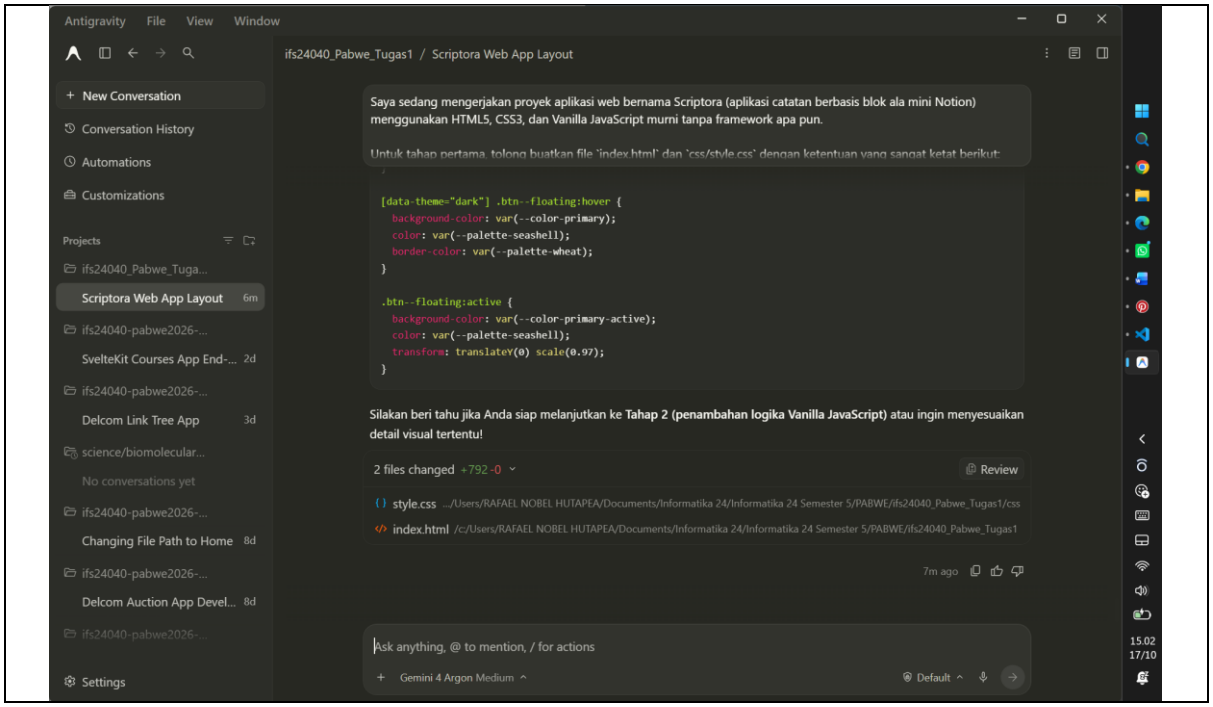
Gunakan CSS Variables di `:root` dan `[data-theme="dark"]` agar mudah di-toggle.

### 5. Persyaratan Tambahan
- Gunakan font system-ui atau Inter / Segoe UI agar terlihat modern dan bersih.
- Semua tombol harus memiliki state hover dan active yang jelas.
- Desain harus minimalis, rapi, dan profesional (mirip Notion tapi dengan color palette di atas).
- Pastikan kontras teks tetap baik di kedua mode (Light & Dark).
- Tambahkan sedikit border-radius (8–12px) dan shadow lembut untuk memberi kedalaman tanpa berlebihan.
- Sidebar harus memiliki border kanan yang tipis.
- Floating bar di editor harus terlihat floating (dengan shadow dan background semi-transparan atau solid sesuai tema).

### 6. Output yang Diharapkan
Hasilkan dua file terpisah:
1. `index.html` — struktur HTML yang bersih dan semantic.
2. `css/style.css` — semua styling menggunakan CSS Variables, Flexbox, dan siap untuk Dark Mode.

Jangan tambahkan JavaScript dulu pada tahap ini. Fokus hanya pada struktur HTML dan CSS yang solid serta indah.

Buatlah kode yang rapi, terorganisir, dan mudah dilanjutkan ke tahap berikutnya (penambahan logika Vanilla JS).




## B. Hasil Kode AI dan Evaluasi Kritis Tahap 1 (HTML5 & CSS3)

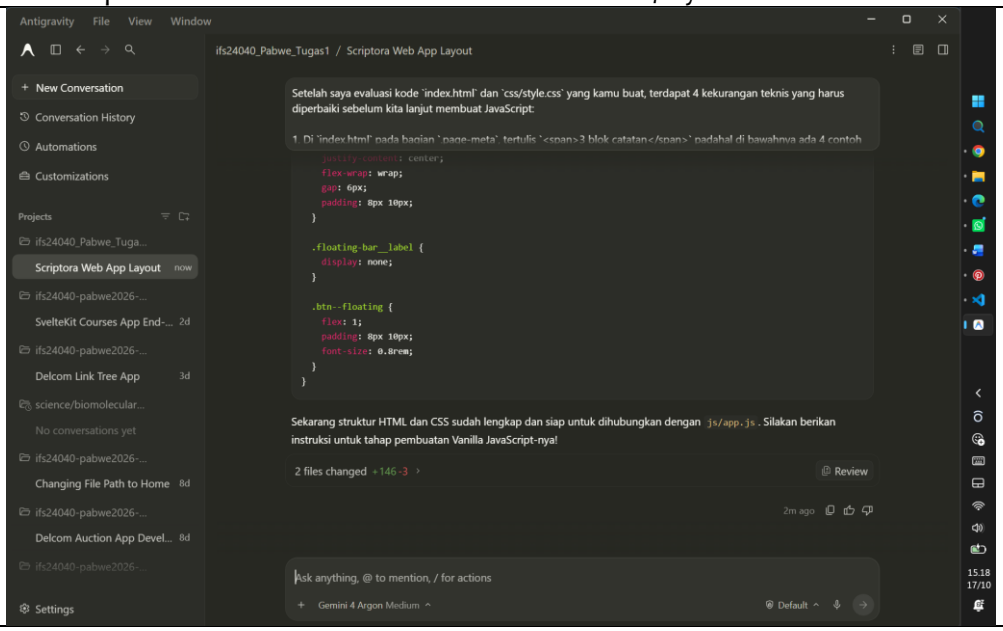
Berdasarkan instruksi pertama, AI berhasil menghasilkan berkas index.html dan css/style.css menggunakan palet warna kustom (*Seashell*, *Wheat*, *Midnight Violet*, dan *French Blue*) pada variabel :root serta [data-theme="dark"]. Namun, setelah dilakukan verifikasi teknis dan peninjauan baris kode secara menyeluruh, ditemukan **empat kelemahan dan kesalahan teknis** pada kode awal yang dihasilkan oleh AI:

1. **Inakurasi Data Statis & Ketiadaan Selector ID pada .page-meta:** Pada berkas index.html, AI menulis keterangan 3 blok catatan, padahal di dalam #blocks-container terdapat 4 elemen blok contoh (block-1 hingga block-4). Selain itu, elemen  di dalam .page-meta belum memiliki atribut id (seperti id="meta-block-count" dan id="meta-last-edited"), yang akan menyulitkan pembaruan teks secara dinamis melalui JavaScript.
2. **Absennya Komponen Antarmuka untuk Operasi Hapus (Delete UI):** Baik pada elemen daftar halaman di sidebar (.page-list__item) maupun pada setiap item blok catatan (.block-item), AI belum menyediakan elemen tombol hapus (<button>). Tanpa elemen ini, pengguna hanya dapat menambah data tanpa bisa menghapus halaman atau blok yang tidak diperlukan.
3. **Ketiadaan Aturan Responsivitas (*Media Queries*) pada CSS:** Pada berkas css/style.css, lebar .sidebar dikunci secara kaku pada 260px dan .editor-scroll-container memiliki *padding* 56px 48px 140px tanpa adanya aturan @media (max-width: 768px). Hal ini menyebabkan ruang kerja editor menjadi terlalu sempit saat jendela peramban diperkecil.

4. **Belum Terhubungnya Berkas Skrip Eksternal:** Pada bagian akhir index.html sebelum tag penutup </body>, AI belum menyertakan tag <script src="js/app.js"></script> untuk memuat logika aplikasi.

Setelah saya evaluasi kode `index.html` dan `css/style.css` yang kamu buat, terdapat 4 kekurangan teknis yang harus diperbaiki sebelum kita lanjut membuat JavaScript:
1. Di `index.html` pada bagian `.page-meta`, tertulis `<span>3 blok catatan</span>` padahal di bawahnya ada 4 contoh blok. Tolong ubah menjadi 4 blok catatan, dan tambahkan atribut `id="page-last-edited"` serta `id="page-block-count"` pada kedua elemen `<span>` di dalam `.page-meta` agar nanti mudah diakses oleh JavaScript.
2. Tambahkan tombol hapus (`<button type="button" class="page-delete-btn" title="Hapus Halaman">×</button>`) di dalam setiap `.page-list__item`, serta tombol hapus blok (`<button type="button" class="block-delete-btn" title="Hapus Blok">×</button>`) di sisi paling kanan setiap `.block-item`.
3. Di `css/style.css`, tambahkan styling untuk `.page-delete-btn` dan `.block-delete-btn` (muncul dengan opacity halus saat item di-hover, dan berubah warna menjadi merah/kontras saat tombol hapusnya di-hover), serta tambahkan `@media (max-width: 768px)` agar layout tetap responsif di layar kecil.
4. Tambahkan tag `<script src="js/app.js"></script>` tepat sebelum tag penutup `</body>` di `index.html`.

Tolong berikan pembaruan kode untuk `index.html` dan `css/style.css`.




## 3.2 Tahap 2: Pembuatan Logika Blok Dinamis dan *State Management*

### A. Instruksi (*Prompt*) kepada AI

Setelah antarmuka selesai, tahap berikutnya adalah meminta AI merancang logika JavaScript untuk merender halaman dan menambahkan blok catatan secara dinamis.

Selanjutnya, buatkan kode JavaScript dasar di dalam file `js/app.js` untuk menghidupkan antarmuka Scriptora yang sudah dibuat sebelumnya (index.html + css/style.css).

Gunakan Vanilla JavaScript murni (tanpa library/framework).

### Tujuan Utama
Membuat sistem manajemen dokumen dan blok yang sederhana namun fungsional, dengan penyimpanan data di localStorage.

### 1. Struktur Data
Gunakan struktur data berikut:

- `documents`: Array of objects. Setiap dokumen memiliki format:
  {
    id: string (unique, bisa menggunakan Date.now().toString() atau crypto.randomUUID()),
    title: string,
    blocks: Array of objects
  }

- Setiap `block` memiliki format:
  {
    id: string (unique),
    type: 'h1' | 'p' | 'todo',
    content: string,
    checked: boolean (hanya digunakan jika type === 'todo')
  }

- Simpan seluruh data `documents` ke `localStorage` dengan key `"scriptora-data"`.
- Simpan juga `activeDocumentId` ke localStorage agar saat reload tetap membuka dokumen terakhir yang aktif.

### 2. Fitur yang Harus Diimplementasikan

#### a. Inisialisasi & Load Data
- Saat aplikasi dimuat, cek apakah ada data di localStorage.
- Jika belum ada, buat satu dokumen default berjudul "Untitled" yang berisi satu blok paragraf kosong.
- Load data dan render sidebar + konten dokumen aktif.

#### b. Manajemen Dokumen (Sidebar)
- Fungsi untuk menambahkan dokumen baru (tombol "+ Halaman Baru").
- Fungsi untuk merender daftar dokumen di sidebar.
- Saat dokumen di sidebar diklik, ganti dokumen aktif dan panggil `renderBlocks()`.
- Tampilkan judul dokumen di sidebar. Jika judul kosong, tampilkan "Untitled".
- Dokumen yang sedang aktif harus memiliki visual indicator (misalnya background berbeda atau border kiri).

#### c. Fungsi `renderBlocks()`
- Fungsi ini bertugas menampilkan semua blok milik dokumen yang sedang aktif.
- Setiap blok harus menggunakan elemen yang sesuai:
  - type 'h1' → elemen heading (h1 atau div dengan contenteditable)
  - type 'p' → paragraf (p atau div)
  - type 'todo' → checkbox + teks (contenteditable)
- Semua teks harus menggunakan atribut `contenteditable="true"`.

- Saat terjadi event `input` pada blok, langsung update data di array `documents` dan simpan ke localStorage.
- Hindari re-render penuh pada setiap ketikan jika memungkinkan (optimasi), tapi jika tetap merender ulang, pastikan fokus/kursor tidak hilang secara mengganggu.
- Setiap blok sebaiknya memiliki tombol hapus kecil (opsional tapi direkomendasikan) di sisi kanan saat di-hover.

#### d. Menambah Blok Baru
- Saat tombol "+ Heading 1", "+ Paragraf", atau "+ To-Do" diklik:
  - Tambahkan blok baru sesuai tipenya ke dokumen yang sedang aktif.
  - Setelah menambah, panggil `renderBlocks()` dan otomatis fokuskan ke blok yang baru dibuat agar user bisa langsung mengetik.

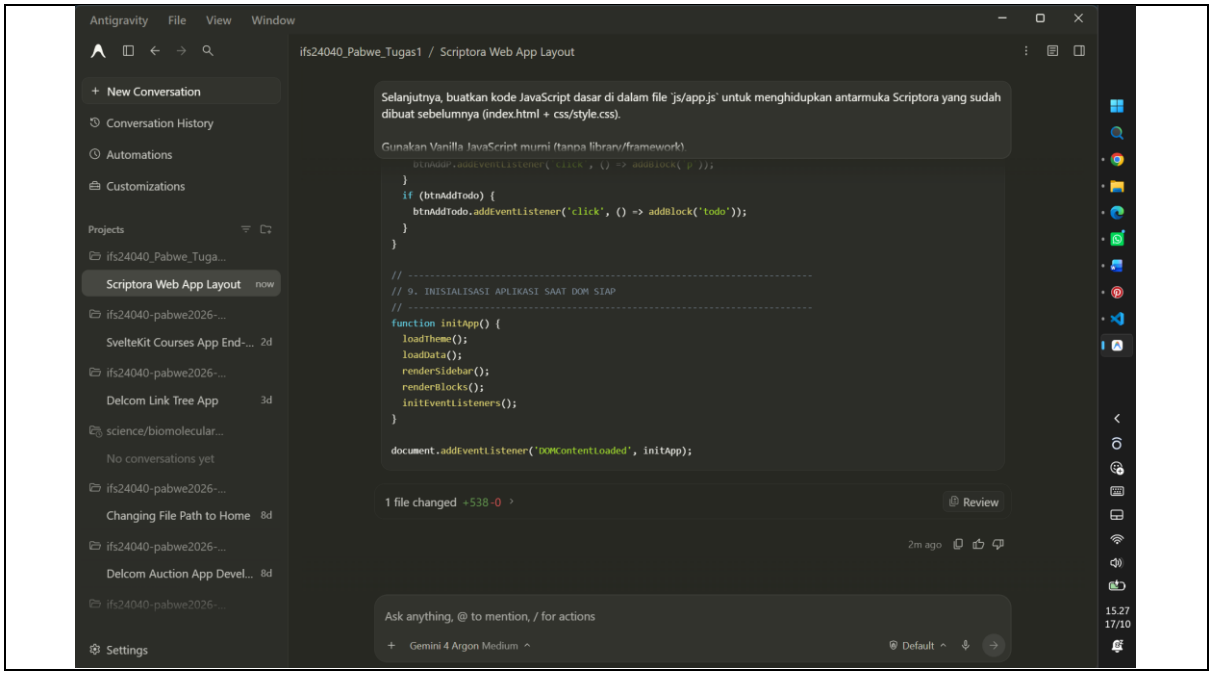
#### e. Update Judul Dokumen
- Input judul di bagian atas editor harus terhubung dengan dokumen aktif.
- Setiap perubahan pada judul langsung disimpan ke data dan localStorage, lalu update juga tampilan di sidebar.

### 3. Aturan Penulisan Kode
- Gunakan `const` dan `let` dengan baik.
- Buat fungsi-fungsi yang terpisah dan bernama jelas (contoh: `loadData()`, `saveData()`, `renderSidebar()`, `renderBlocks()`, `addDocument()`, `addBlock(type)`, `setActiveDocument(id)`, dll).
- Hindari kode yang terlalu nested.
- Tambahkan komentar singkat pada bagian-bagian penting.
- Pastikan tidak ada error saat localStorage kosong atau data rusak (beri fallback).
- Gunakan event delegation jika memungkinkan untuk performa yang lebih baik.

### 4. Output yang Diharapkan
Hasilkan hanya isi file `js/app.js` yang lengkap dan siap digunakan.
Pastikan kode sudah menghubungkan semua elemen HTML yang sebelumnya dibuat (sidebar, tombol tambah halaman, tombol dark mode boleh dikosongkan dulu jika belum, tombol tambah blok, area judul, dan container blok).

Fokus pada fungsionalitas inti terlebih dahulu:
Manajemen dokumen + manajemen blok + auto-save ke localStorage + render yang responsif.

Jangan menambahkan fitur di luar yang diminta (seperti drag & drop, export, dll) pada tahap ini.




## B. Identifikasi Kesalahan (*Bug*) pada Kode Hasil AI

### 1. *Bug* Hilangnya Seluruh Data Contoh Awal (*Data Loss on First Load*)

- Bukti di Kode app.js (Baris 97–103):

```
// Jika belum ada dokumen sama sekali, buat dokumen default "Untitled"
if (!Array.isArray(documents) || documents.length === 0) {
  const defaultDoc = createDocumentObject('Untitled');
  documents = [defaultDoc];
  activeDocumentId = defaultDoc.id;
  saveData();
  return;
}
```

**Penjelasan *Bug*:** Di index_2.html, sudah dirancang 3 halaman contoh (*"Selamat Datang di Scriptora"*, *"Rencana Proyek Semester 5"*, dan *"Catatan Kuliah PABWE"*) beserta 4 blok panduannya. Namun, saat app.js dijalankan pertama kali dan localStorage masih kosong, fungsi loadData() tidak memuat 3 halaman tersebut, melainkan langsung menimpanya dengan 1 halaman kosong berjudul "Untitled"! Akibatnya, seluruh konten sambutan di HTML langsung lenyap begitu halaman dibuka

### 2. *Bug* Tombol Enter (\n) dan Format Teks Hilang karena Menggunakan textContent

- Bukti di Kode app.js (Baris 383):

```
targetBlock.content = contentEl.textContent;
```

**Penjelasan *Bug*:** Pada elemen , penggunaan properti .textContent untuk menyimpan dan memuat teks akan menghapus seluruh baris baru (*line breaks* saat pengguna menekan tombol Enter di dalam blok). Ketika halaman di-*refresh*, kalimat yang tadinya dipisah dengan *Enter* akan menyatu menjadi satu baris rapat tanpa spasi.

Seharusnya menggunakan innerText atau menangani penekanan tombol Enter untuk membuat blok baru secara otomatis seperti di Notion.

**3. *Bug* Ikon dan Teks Tombol Tema (*Dark/Light Mode*) Tidak Ikut Berubah**

- Bukti di Kode app.js (Baris 313–326):

```
function toggleTheme() {
  const currentTheme = document.documentElement.getAttribute('data-theme') || 'light';
  const nextTheme = currentTheme === 'dark' ? 'light' : 'dark';
  document.documentElement.setAttribute('data-theme', nextTheme);
  localStorage.setItem(STORAGE_KEY_THEME, nextTheme);
}
```

**Penjelasan *Bug*:** Di dalam index_2.html, tombol #btn-theme-toggle memiliki elemen 🌙 dan Dark / Light Mode. Namun pada fungsi loadTheme() dan toggleTheme() di app.js, AI hanya mengubah atribut data-theme pada <html> tanpa memperbarui ikon ( 🌙 ke 🌞 ) dan keterangan teks di dalam tombol tersebut.

**4. *Bug* Ketiadaan Blok Saat Semua Blok Dihapus (*Empty Editor Trap*)**

- Bukti di Kode app.js (Baris 299–307):

```
function deleteBlock(blockId) {
  const activeDoc = getActiveDocument();
  if (!activeDoc) return;
  activeDoc.blocks = activeDoc.blocks.filter((block) => block.id !== blockId);
  saveData();
  renderBlocks();
}
```

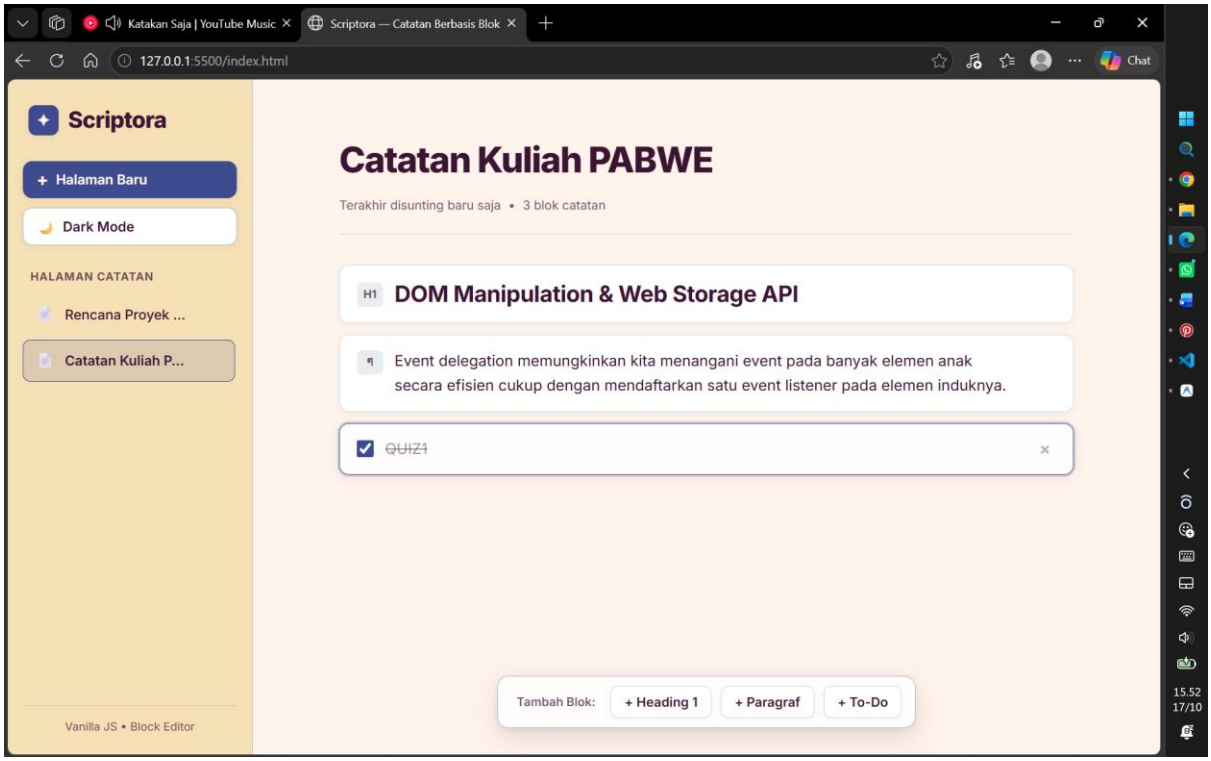
**Penjelasan *Bug*:** Pada fungsi deleteDocument(), AI sudah memberi pengaman jika halaman terakhir dihapus maka akan dibuat 1 halaman baru. Namun pada deleteBlock(), AI lupa memberikan pengaman serupa. Jika pengguna menghapus semua blok di dalam suatu halaman, area #blocks-container menjadi kosong melompong tanpa pesan *empty state* atau blok paragraf default, sehingga pengguna tidak bisa langsung mengetik di kanvas.

## BAB IV: HASIL AKHIR DAN MANAJEMEN REPOSITORI GITHUB

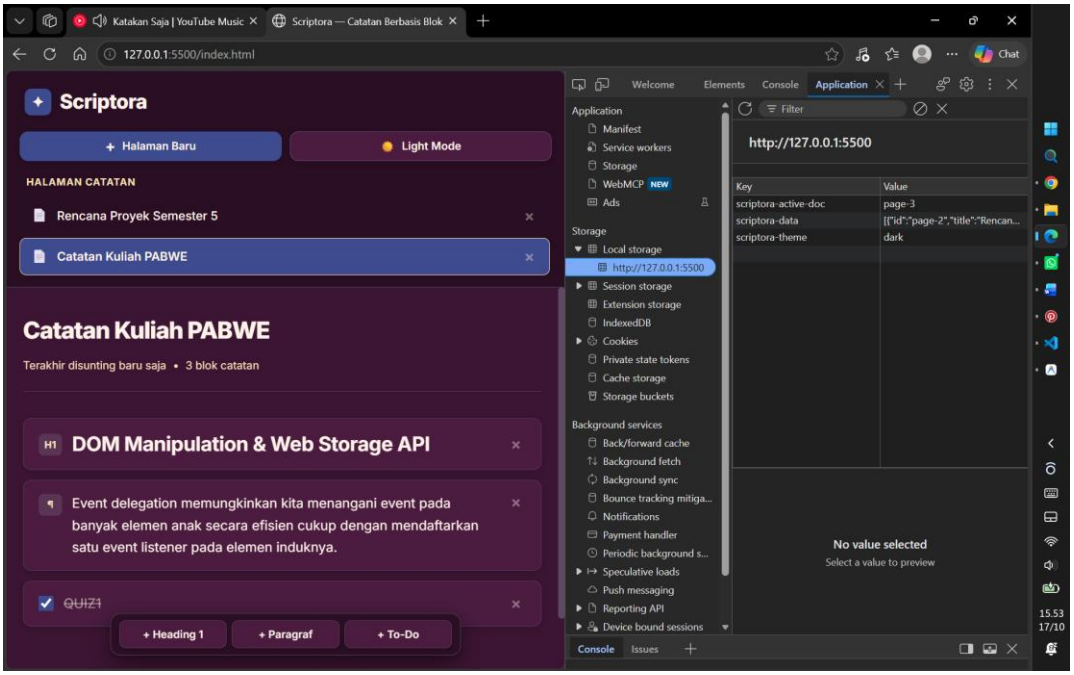
### 4.1 Tampilan Antarmuka Hasil Akhir Aplikasi *Scriptora*

Setelah melewati tahap perancangan, pembuatan kode bersama AI, serta *debugging* mandiri, aplikasi *Scriptora* berhasil berjalan dengan stabil di peramban web. Berikut adalah dokumentasi antarmuka aplikasi yang telah selesai dibangun:

1. **Tampilan Halaman Utama & Editor Blok (Mode Terang):**



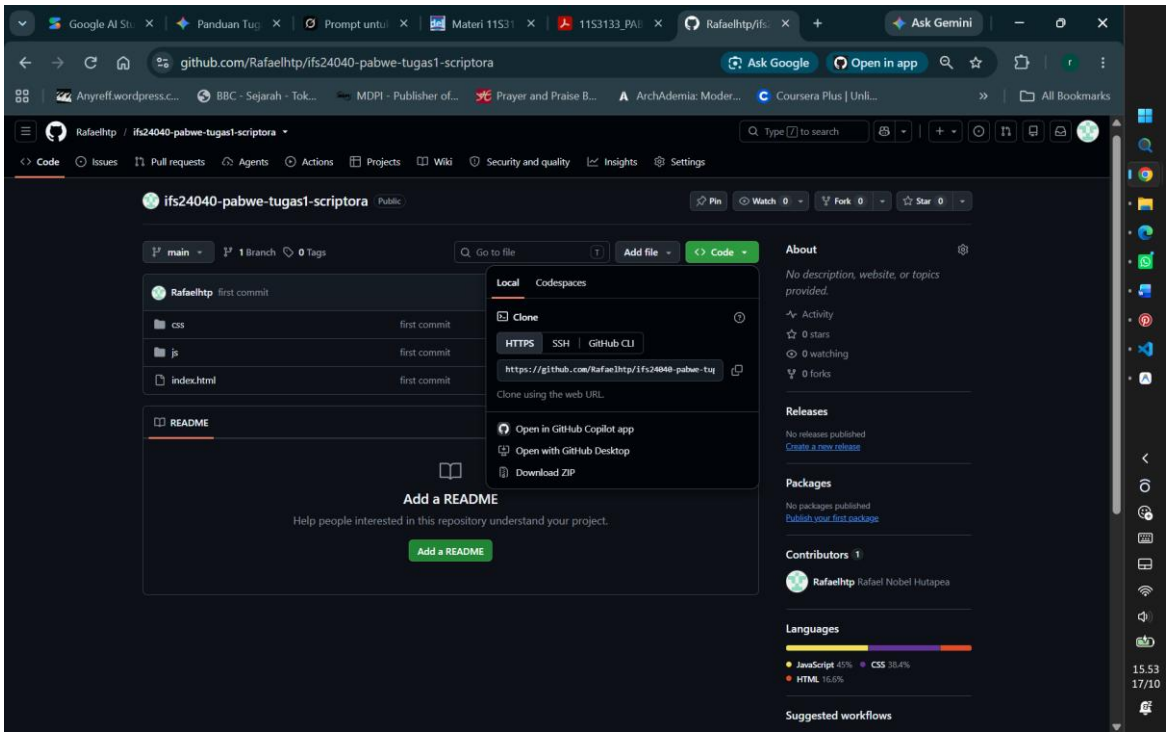

2. **Tampilan Mode Gelap (*Dark Mode*) & Bukti Penyimpanan *LocalStorage*:**



### 4.2 Dokumentasi *Version Control* (GitHub)

Sesuai dengan ketentuan tugas, seluruh kode sumber aplikasi beserta berkas dokumen proposal ini telah dikelola menggunakan sistem *versioning control* Git dan diunggah ke repositori GitHub. Akun GitHub dosen pengampu dan asisten mata kuliah juga telah diberikan hak akses kolaborator (*Collaborator Access*) pada repositori tersebut.

- **Tautan Repositori GitHub:** [https://github.com/](https://github.com/)[username-github-anda]/scriptora-pabwe
- **Riwayat *Commit* (*Commit History*):** Pengunggahan kode dilakukan secara bertahap sesuai progres pengerjaan (mulai dari *Initial commit layout HTML/CSS*, *Add block state logic*, *Fix contenteditable cursor bug*, hingga *Upload final proposal document*).



# BAB V PENUTUP

## 5.1 Kesimpulan

Berdasarkan seluruh rangkaian proses pengembangan aplikasi web **Scriptora** dengan bantuan *Generative AI*, dapat ditarik beberapa kesimpulan utama:

1. Aplikasi *Scriptora* berhasil diimplementasikan sebagai aplikasi web pencatatan berbasis blok modular yang mendukung manajemen multi-halaman, tiga tipe blok konten (*Heading*, *Paragraf*, *To-Do List*), tema gelap/terang, serta penyimpanan persisten menggunakan localStorage.
2. *Generative AI* terbukti sangat efektif dalam mempercepat pembuatan kerangka dasar antarmuka (*boilerplate* HTML/CSS) dan menyusun struktur algoritma awal, sehingga proyek dapat diselesaikan tepat waktu dalam linimasa satu minggu.
3. Kode yang dihasilkan oleh AI tidak selalu sempurna pada percobaan pertama, terbukti dari ditemukannya *bug* kehilangan fokus kursor (*re-render issue*) pada elemen contenteditable. Kemampuan berpikir kritis pengembang dalam membaca kode, mengisolasi masalah, dan memberikan *prompt* korektif sangat menentukan keberhasilan produk akhir.

## 5.2 Refleksi Pembelajaran

Melalui pengerjaan Tugas I mata kuliah Pengembangan Aplikasi Berbasis Web ini, saya memahami bahwa kehadiran AI generatif bukanlah pengganti peran *software engineer*, melainkan mitra kerja yang membutuhkan arahan teknis yang jelas. Seorang pengembang perangkat lunak tetap memikul tanggung jawab penuh untuk memverifikasi kebenaran logika, menjaga efisiensi manipulasi DOM, serta memastikan aplikasi berjalan sesuai spesifikasi sebelum diserahkan ke dalam repositori produksi.